Java Full Stack Interview – Spring & JPA Q&A;

106 interview-ready questions for 5 years of Java Full Stack experience. Format: What is it? Why is it used? How is it used? Examples only where they improve understanding.

SPRING CORE

1. What is Spring Framework?

What is it? Spring is a Java framework used to build enterprise applications. It provides features such as IoC/DI, AOP, transaction management, web development, and data access.

Why is it used? It reduces boilerplate code and helps build applications that are loosely coupled, maintainable, testable, and scalable.

How is it used? We use Spring Container to create/manage beans and inject their dependencies.

2. Why is Spring called a lightweight framework?

What is it? Spring can work with simple Java classes (POJOs) and does not require heavy framework-specific inheritance or a special runtime container.

Why is it used? It makes applications easier to develop, test, deploy, and maintain.

How is it used? Spring uses Dependency Injection and configuration/annotations to manage normal Java objects.

3. What are the advantages of Spring?

What is it? Spring provides infrastructure for enterprise Java applications.

Why is it used? Key advantages include loose coupling, easier testing, transaction management, AOP, database integration, web/MVC support, and technology integration.

How is it used? Use the Spring modules/features required by the application.

4. What is IoC (Inversion of Control)?

What is it? IoC means responsibility for creating and managing objects is transferred from application code to the Spring Container.

Why is it used? It reduces tight coupling because classes do not create their own dependencies.

How is it used? Spring creates beans, resolves dependencies, and manages their lifecycle.

Example: Instead of a class doing new PaymentService(), Spring creates PaymentService and provides it.

5. What is Dependency Injection?

What is it? DI is a technique where a class receives the objects it depends on from an external source instead of creating them itself.

Why is it used? It provides loose coupling and makes code easier to test and change.

How is it used? Spring injects dependencies mainly through constructor, setter, or field injection.

Example: OrderService can receive PaymentService through its constructor instead of creating it with new.

6. Difference between IoC and DI?

What is it? IoC is the principle of transferring control of object creation to the framework. DI is the technique used to implement that principle.

Why is it used? Both reduce coupling and improve maintainability.

How is it used? Spring implements IoC mainly through Dependency Injection.

Example: IoC = principle; DI = mechanism/technique.

7. What are the types of Dependency Injection?

What is it? The commonly discussed types are Constructor Injection, Setter Injection, and Field Injection.

Why is it used? They provide different ways of supplying dependencies to a class.

How is it used? Spring can inject through constructor, setter, or field. Constructor injection is generally preferred.

8. Constructor Injection vs Setter Injection?

What is it? Constructor Injection supplies dependencies through the constructor; Setter Injection supplies them through a setter method.

Why is it used? Constructor injection is better for mandatory dependencies; setter injection can suit optional dependencies.

How is it used? Define mandatory dependencies as constructor parameters and optional ones through setters.

9. Which injection is preferred and why?

What is it? Constructor Injection is generally preferred.

Why is it used? It guarantees mandatory dependencies, supports final fields, improves testability, makes dependencies explicit, and helps immutability.

How is it used? Define dependencies as constructor parameters; Spring injects them automatically.

10. What is loose coupling?

What is it? Loose coupling means a class depends on an abstraction/interface rather than a specific implementation.

Why is it used? The implementation can be changed with minimal changes to the dependent class.

How is it used? Use interfaces and Dependency Injection.

11. What is tight coupling?

What is it? Tight coupling means one class is strongly dependent on a specific implementation or another class.

Why is it used? It makes the application harder to modify, test, and maintain.

How is it used? It commonly happens when a class directly creates another class using new.

12. How does Spring achieve loose coupling?

What is it? Spring achieves loose coupling primarily through Dependency Injection and programming to interfaces.

Why is it used? Classes do not need to know how their dependencies are created.

How is it used? The container creates the implementation and injects it into the dependent class.

13. What is a Spring Bean?

What is it? A Spring Bean is an object created, configured, and managed by the Spring Container.

Why is it used? Spring can manage its dependencies, lifecycle, configuration, and scope.

How is it used? Register beans with @Component, @Service, @Repository, or @Bean.

14. What is Spring Container?

What is it? The Spring Container creates, configures, injects, and manages Spring Beans.

Why is it used? It provides Spring's IoC functionality.

How is it used? It reads configuration/annotations, creates beans, resolves dependencies, and manages lifecycle. Main containers are BeanFactory and ApplicationContext.

15. BeanFactory vs ApplicationContext?

What is it? Both are IoC containers; ApplicationContext is more feature-rich.

Why is it used? BeanFactory provides basic bean management, while ApplicationContext adds features such as events, resource loading, and i18n.

How is it used? Modern Spring Boot applications generally use ApplicationContext.

16. What is Autowiring?

What is it? Autowiring means Spring automatically identifies and injects a suitable dependency.

Why is it used? It avoids manually creating and connecting dependent objects.

How is it used? Spring uses type matching and, when needed, @Qualifier or @Primary.

17. What is @Autowired?

What is it? @Autowired tells Spring to inject a suitable dependency automatically.

Why is it used? It simplifies dependency wiring.

How is it used? It can be used on constructors, setters, or fields. A single constructor does not need @Autowired in modern Spring.

18. What is @Qualifier?

What is it? @Qualifier specifies exactly which bean should be injected when multiple beans of the same type exist.

Why is it used? It resolves ambiguity.

How is it used? Use @Qualifier with the bean name on the injection point.

19. What is @Primary?

What is it? @Primary marks one bean as the preferred/default bean when multiple beans of the same type exist.

Why is it used? It avoids ambiguity when one implementation should normally be selected.

How is it used? Put @Primary on the preferred bean.

20. What happens when multiple beans of the same type exist?

What is it? Spring finds multiple candidates and cannot decide which one to inject.

Why is it used? An unqualified dependency normally needs a single suitable bean.

How is it used? Resolve it using @Qualifier or @Primary.

21. What exception occurs when multiple beans match?

What is it? Spring throws NoUniqueBeanDefinitionException.

Why is it used? More than one bean matches and Spring cannot choose one.

How is it used? Use @Qualifier or @Primary.

22. What is a stereotype annotation?

What is it? A stereotype annotation identifies a class as a particular type of Spring-managed component.

Why is it used? It helps Spring discover and register classes as beans.

How is it used? Common stereotypes are @Component, @Service, @Repository, and @Controller.

23. What is @Component?

What is it? @Component marks a general-purpose class as a Spring-managed bean.

Why is it used? It allows Spring to discover and manage the class automatically.

How is it used? Component scanning detects it.

24. What is @Service?

What is it? @Service is a specialized @Component used for the business/service layer.

Why is it used? It clearly communicates that the class contains business logic.

How is it used? Annotate the service class with @Service.

25. What is @Repository?

What is it? @Repository is a specialized component for the data-access/persistence layer.

Why is it used? It identifies DAO/repository components and enables persistence exception translation.

How is it used? Use it on classes responsible for database access.

26. What is @Controller?

What is it? @Controller marks a class as a Spring MVC controller that handles web requests.

Why is it used? It separates HTTP request handling from business and data-access logic.

How is it used? Map requests using @GetMapping, @PostMapping, etc.

27. Difference between @Component and @Service?

What is it? Both register Spring Beans; @Component is generic, while @Service communicates business/service responsibility.

Why is it used? It improves semantic clarity and layer separation.

How is it used? Use @Component for generic components and @Service for business logic.

28. Difference between @Service and @Repository?

What is it? @Service represents business logic; @Repository represents data-access logic.

Why is it used? They make layer responsibilities clear. @Repository also participates in persistence exception translation.

How is it used? Typical flow: Controller → Service → Repository → Database.

29. What is @ComponentScan?

What is it? @ComponentScan tells Spring which packages to scan for component classes.

Why is it used? Spring needs to discover @Component, @Service, @Repository, @Controller, etc.

How is it used? It scans specified packages and registers detected components as beans.

30. What is Bean Lifecycle?

What is it? Bean lifecycle describes the stages a bean goes through from creation to destruction.

Why is it used? It lets us perform custom initialization and cleanup.

How is it used? Simplified: create → inject dependencies → initialize → ready → destroy.

31. What is @PostConstruct?

What is it? @PostConstruct marks a method that runs after dependency injection and before the bean is used.

Why is it used? Useful for initialization requiring injected dependencies.

How is it used? Annotate an initialization method with @PostConstruct.

32. What is @PreDestroy?

What is it? @PreDestroy marks a method that runs before a bean is destroyed.

Why is it used? Useful for cleanup such as releasing resources.

How is it used? Annotate a cleanup method with @PreDestroy.

33. What is eager initialization?

What is it? Eager initialization means a bean is created when the ApplicationContext starts instead of when first requested.

Why is it used? The bean is immediately ready, though startup work can increase.

How is it used? Singleton beans are eager by default in an ApplicationContext unless configured otherwise.

34. What is lazy initialization?

What is it? Lazy initialization means a bean is created only when it is first needed/requested.

Why is it used? It can reduce startup time and avoid creating unused beans.

How is it used? Use @Lazy or configure lazy initialization.

35. What is @Lazy?

What is it? @Lazy tells Spring to delay bean initialization until the bean is actually needed.

Why is it used? Useful for expensive-to-create or rarely used beans.

How is it used? Apply @Lazy to the bean or configuration.

36. What is @Configuration?

What is it? @Configuration marks a class as a source of Spring bean definitions and application configuration.

Why is it used? It allows explicit Java-based configuration.

How is it used? Methods inside can be annotated with @Bean.

37. What is @Bean?

What is it? @Bean tells Spring to register the object returned by a method as a Spring Bean.

Why is it used? Useful for configuring third-party classes or classes you cannot annotate.

How is it used? Define the object in a @Configuration class.

38. XML Configuration vs Java Configuration?

What is it? Both configure Spring beans. XML stores definitions in XML; Java configuration uses @Configuration and @Bean.

Why is it used? Java configuration provides type safety and keeps configuration in Java code.

How is it used? Modern Spring applications generally prefer annotation/Java-based configuration.

39. What is @Profile?

What is it? @Profile makes a bean or configuration active only when a particular Spring profile is active.

Why is it used? Different environments need different configurations.

How is it used? Use profiles such as dev, test, and prod.

40. Why do we use Profiles?

What is it? Profiles allow environment-specific configurations.

Why is it used? Development, testing, staging, and production may use different databases, URLs, and settings.

How is it used? Activate the required profile for the environment.

41. What is @Value?

What is it? @Value injects a value from configuration/properties into a Spring-managed object.

Why is it used? It externalizes configuration instead of hard-coding values.

How is it used? Use @Value("${property.name}") on an injection point.

42. What is @PropertySource?

What is it? @PropertySource tells Spring to load properties from a specified properties file.

Why is it used? It supports custom property files.

How is it used? Use @PropertySource("classpath:custom.properties") in configuration.

SPRING BOOT

43. What is Spring Boot?

What is it? Spring Boot is built on Spring and simplifies application development with auto-configuration, starters, embedded servers, and production-ready features.

Why is it used? It reduces configuration and speeds up development.

How is it used? Create a Boot application, add starters, configure properties, and run it directly.

44. Advantages of Spring Boot?

What is it? Spring Boot simplifies Spring development.

Why is it used? Key benefits include auto-configuration, starters, embedded servers, minimal configuration, easy deployment, Actuator, and externalized configuration.

How is it used? Add required starters and Boot configures many common components.

45. What is Auto Configuration?

What is it? Auto-configuration automatically configures application components based mainly on classpath dependencies and configuration properties.

Why is it used? It avoids manually configuring common infrastructure.

How is it used? @EnableAutoConfiguration enables this feature.

46. What is a Starter Dependency?

What is it? A starter is a convenient dependency that groups commonly required libraries for a feature.

Why is it used? One dependency can replace manually adding many related libraries.

How is it used? Example: spring-boot-starter-web for common web application dependencies.

47. What is Embedded Tomcat?

What is it? Embedded Tomcat means Tomcat runs inside the Spring Boot application.

Why is it used? It simplifies deployment because an external Tomcat installation is usually unnecessary.

How is it used? Package the application as a JAR and run it with java -jar.

48. What is @SpringBootApplication?

What is it? @SpringBootApplication is the main annotation used on a Spring Boot application's entry-point class.

Why is it used? It enables core Boot features.

How is it used? It combines @Configuration, @EnableAutoConfiguration, and @ComponentScan.

49. Which annotations are present inside @SpringBootApplication?

What is it? It combines three annotations: @Configuration, @EnableAutoConfiguration, and @ComponentScan.

Why is it used? Together they provide Java configuration, Boot auto-configuration, and component scanning.

How is it used? Remember: SpringBootApplication = Configuration + AutoConfiguration + ComponentScan.

50. What is application.properties?

What is it? It is a Spring Boot configuration file for application-level settings.

Why is it used? It lets configuration change without modifying Java source code.

How is it used? Store settings such as server.port, datasource properties, and application name.

51. application.properties vs application.yml?

What is it? Both are formats for externalized Spring Boot configuration.

Why is it used? YAML is often easier to read for nested/hierarchical configuration.

How is it used? Choose either based on project conventions.

52. How do you change the server port?

What is it? Spring Boot commonly uses port 8080 by default.

Why is it used? Change it when another application uses that port or a different port is required.

How is it used? Set server.port=8081 in application.properties or the equivalent YAML.

53. What is dependency management?

What is it? Dependency management means controlling the libraries and compatible versions used by the application.

Why is it used? It prevents version conflicts and reduces manual version declarations.

How is it used? Spring Boot provides dependency management through its parent/BOM.

54. What is Spring Boot Actuator?

What is it? Actuator provides production-ready monitoring and management features.

Why is it used? It helps monitor health, metrics, and runtime information.

How is it used? Add Actuator and expose required endpoints such as /actuator/health.

JPA

55. What is JPA?

What is it? JPA (Java Persistence API) is a Java specification defining a standard way to map Java objects to relational database data and perform persistence operations.

Why is it used? It provides a standard persistence programming model independent of a particular ORM provider.

How is it used? Use JPA annotations and APIs with an implementation such as Hibernate.

56. Is JPA a framework?

What is it? No. JPA is a specification/API.

Why is it used? It defines standard interfaces, annotations, and behavior.

How is it used? A provider such as Hibernate implements it.

57. Is JPA an implementation?

What is it? No. JPA is the standard; it does not implement the persistence engine.

Why is it used? An implementation is required to perform database persistence.

How is it used? Common implementations include Hibernate and EclipseLink.

58. What is ORM?

What is it? ORM (Object-Relational Mapping) maps Java objects/classes to relational database tables.

Why is it used? It lets developers work mainly with objects instead of writing SQL for every operation.

How is it used? Class→Table, Object→Row, Field→Column, Relationship→database relationship.

59. What is Hibernate?

What is it? Hibernate is a popular ORM framework and JPA implementation.

Why is it used? It simplifies persistence by mapping Java objects to tables and handling SQL generation/persistence.

How is it used? Use Hibernate directly or through JPA APIs; Spring Boot commonly uses it as a JPA provider.

60. Hibernate vs JPA?

What is it? JPA is a specification; Hibernate is an implementation/framework of that specification.

Why is it used? JPA gives a standard API while Hibernate performs the actual persistence work.

How is it used? Application → JPA API → Hibernate → Database.

61. What is EntityManager?

What is it? EntityManager is a JPA interface used to manage entity objects and perform persistence operations.

Why is it used? It provides operations such as persist, find, merge, and remove.

How is it used? Use entityManager.persist(), find(), merge(), or remove().

62. What is Persistence Context?

What is it? A Persistence Context is a set of entity instances currently managed by an EntityManager.

Why is it used? It tracks entity state and synchronizes managed changes with the database; it also provides first-level caching.

How is it used? Entity states include Transient, Managed, Detached, and Removed.

63. What is @Entity?

What is it? @Entity marks a Java class as a JPA entity that can be persisted.

Why is it used? It tells JPA to include the class in ORM mapping.

How is it used? Annotate the persistent domain class with @Entity.

64. What is @Table?

What is it? @Table specifies the database table associated with an entity.

Why is it used? Useful when the table name differs or table-level configuration is needed.

How is it used? Use @Table(name="users") on the entity.

65. What is @Id?

What is it? @Id marks a field/property as the entity's primary key.

Why is it used? JPA needs a unique identifier to manage each entity.

How is it used? Annotate the ID field with @Id.

66. What is @GeneratedValue?

What is it? @GeneratedValue tells JPA to generate the primary-key value automatically.

Why is it used? It avoids manually assigning IDs for new entities.

How is it used? Use a strategy such as GenerationType.IDENTITY, depending on the database.

67. What is @Column?

What is it? @Column customizes mapping between an entity field/property and a database column.

Why is it used? It can configure name, nullability, length, uniqueness, etc.

How is it used? Use @Column(name="user_name", nullable=false).

68. What is persist()?

What is it? persist() makes a new entity managed by the persistence context.

Why is it used? It is used to make a new entity persistent.

How is it used? entityManager.persist(user); the insert is synchronized on flush/commit.

69. What is merge()?

What is it? merge() copies the state of a detached or new entity into a managed entity.

Why is it used? It is commonly used when detached state needs to be associated with the current persistence context.

How is it used? User managed = entityManager.merge(user); the returned object is the managed instance.

70. What is remove()?

What is it? remove() marks a managed entity for deletion.

Why is it used? It is used to delete an entity.

How is it used? entityManager.remove(user); deletion is synchronized on flush.

71. Difference between persist() and merge()?

What is it? persist() mainly makes a new entity managed; merge() copies state into a managed instance and returns that managed instance.

Why is it used? They have different entity-state semantics.

How is it used? Use persist for new entities; merge is common with detached state.

72. Difference between save() and persist()?

What is it? persist() is a standard JPA EntityManager operation; save() is a Spring Data repository method.

Why is it used? save() provides a convenient abstraction over JPA persistence operations.

How is it used? Spring Data's save generally decides whether the entity is new or existing and delegates appropriately.

73. What is @OneToOne?

What is it? @OneToOne represents a relationship where one entity is associated with one other entity.

Why is it used? Use it for one-to-one business relationships.

How is it used? Example: one person has one passport.

74. What is @OneToMany?

What is it? @OneToMany represents one entity associated with multiple entities.

Why is it used? Useful for parent-to-many-child relationships.

How is it used? Example: one department has many employees.

75. What is @ManyToOne?

What is it? @ManyToOne represents many entities associated with one entity.

Why is it used? Commonly used for foreign-key relationships.

How is it used? Example: many employees belong to one department.

76. What is @ManyToMany?

What is it? @ManyToMany represents many entities associated with many entities.

Why is it used? Use it when both sides can have multiple relationships.

How is it used? Example: students and courses; a join table is commonly used.

77. What is Cascade?

What is it? Cascade defines which persistence operations on one entity should propagate to related entities.

Why is it used? It reduces the need to perform the same operations separately on child entities.

How is it used? Configure PERSIST, MERGE, REMOVE, or ALL as required.

78. What is FetchType?

What is it? FetchType determines when related entity data is loaded.

Why is it used? It controls loading behavior and can strongly affect performance.

How is it used? Use LAZY or EAGER.

79. Eager Fetch vs Lazy Fetch?

What is it? EAGER loads related data immediately; LAZY loads it when the relationship is accessed.

Why is it used? Lazy loading can avoid unnecessary data retrieval and is generally preferable for large collections.

How is it used? Use fetch=FetchType.LAZY when appropriate.

80. Default fetch type of OneToMany?

What is it? The JPA default is LAZY.

Why is it used? Collections are normally not loaded until accessed.

How is it used? OneToMany → LAZY.

81. Default fetch type of ManyToOne?

What is it? The JPA default is EAGER.

Why is it used? The related entity is intended to be fetched eagerly unless configured otherwise.

How is it used? ManyToOne → EAGER; many applications explicitly choose LAZY when appropriate.

SPRING DATA JPA

82. What is Spring Data JPA?

What is it? Spring Data JPA provides a repository abstraction on top of JPA.

Why is it used? It greatly reduces boilerplate database-access code.

How is it used? Create repository interfaces; Spring Data provides the implementation at runtime.

83. Difference between JPA and Spring Data JPA?

What is it? JPA is the persistence specification/API; Spring Data JPA is a Spring abstraction that simplifies working with JPA.

Why is it used? Spring Data adds repository abstractions and convenient query mechanisms.

How is it used? Spring Data JPA → JPA → provider such as Hibernate → Database.

84. What is Repository?

What is it? A Repository is an abstraction responsible for accessing and managing application data.

Why is it used? It separates persistence logic from business logic and reduces boilerplate.

How is it used? Create an interface extending a Spring Data repository interface.

85. What is CrudRepository?

What is it? CrudRepository provides basic Create, Read, Update, and Delete operations.

Why is it used? It gives common data operations without manually implementing them.

How is it used? Extend CrudRepository<Entity, IdType>.

86. What is PagingAndSortingRepository?

What is it? PagingAndSortingRepository provides repository functionality with pagination and sorting support.

Why is it used? It helps retrieve large datasets in pages and ordered results.

How is it used? Use Pageable and Sort with repository methods. Interface inheritance can vary by Spring Data version.

87. What is JpaRepository?

What is it? JpaRepository is a Spring Data JPA repository interface providing CRUD, pagination, sorting, and additional JPA-related functionality.

Why is it used? It is convenient for most Spring Boot applications using JPA.

How is it used? Extend JpaRepository<Entity, IdType>.

88. JpaRepository vs CrudRepository?

What is it? CrudRepository focuses on basic CRUD; JpaRepository provides CRUD plus paging, sorting, and additional JPA-specific features.

Why is it used? JpaRepository offers a broader API for JPA applications.

How is it used? Choose based on the repository features required.

89. Why do we create Repository Interfaces?

What is it? They define a data-access contract without requiring developers to write the implementation.

Why is it used? They reduce boilerplate and separate persistence from business logic.

How is it used? Extend JpaRepository, CrudRepository, etc.

90. Who implements JpaRepository methods?

What is it? Spring Data JPA provides the implementation as a runtime-generated proxy.

Why is it used? Developers only define the repository interface for standard operations.

How is it used? Spring creates the proxy and injects it where required.

91. What is save()?

What is it? save() is a Spring Data repository method used to persist a new entity or update an existing entity, depending on whether it is considered new.

Why is it used? It provides a convenient abstraction over JPA persistence operations.

How is it used? userRepository.save(user).

92. What is findById()?

What is it? findById() retrieves an entity using its primary key.

Why is it used? It is used when a specific record is needed by ID.

How is it used? It returns Optional<T> in Spring Data repositories.

93. What is findAll()?

What is it? findAll() retrieves all entities available through the repository.

Why is it used? Useful when the complete result set is required, but large datasets should generally use pagination.

How is it used? userRepository.findAll().

94. What is deleteById()?

What is it? deleteById() deletes an entity using its primary key.

Why is it used? It provides a simple repository delete operation.

How is it used? userRepository.deleteById(id).

95. What is a Derived Query Method?

What is it? A derived query method is a repository method where Spring Data derives the query from the method name.

Why is it used? It avoids writing JPQL/SQL for many common queries.

How is it used? Examples: findByName, findByEmail, findByAgeGreaterThan.

96. How does findByName() work?

What is it? Spring parses findByName, identifies the entity's name property, and creates the corresponding query.

Why is it used? It provides a simple search without explicitly writing a query.

How is it used? Conceptually it becomes a query filtering rows/entities where name equals the parameter.

97. What is @Query?

What is it? @Query lets us explicitly define a JPQL or native SQL query for a repository method.

Why is it used? It is useful for complex queries that are inconvenient as derived method names.

How is it used? Use @Query with JPQL by default or nativeQuery=true for SQL.

98. JPQL vs Native Query?

What is it? JPQL works with entities and their fields; native SQL works directly with database tables and columns.

Why is it used? JPQL is more portable; native SQL is useful for database-specific features or complex SQL.

How is it used? JPQL: SELECT u FROM User u. Native: SELECT * FROM users.

99. What is Pagination?

What is it? Pagination means retrieving a large dataset one page at a time instead of loading everything.

Why is it used? It improves memory usage, response size, and API performance.

How is it used? Use Pageable and Page, e.g. PageRequest.of(page, size).

100. How is Sorting implemented?

What is it? Sorting means retrieving records in a specified order.

Why is it used? It lets APIs and UIs return data in a useful order.

How is it used? Spring Data provides Sort and Pageable for sorting and combined pagination.

TRANSACTIONS

101. What is a Transaction?

What is it? A transaction is a logical unit of database work in which related operations are treated as one unit.

Why is it used? It prevents partial updates and helps maintain consistent data.

How is it used? Successful work is committed; failure can cause rollback.

Example: A money transfer typically treats debit and credit as one transaction.

102. What are ACID Properties?

What is it? ACID stands for Atomicity, Consistency, Isolation, and Durability.

Why is it used? Atomicity means all-or-nothing; Consistency keeps data valid; Isolation controls interference between concurrent transactions; Durability makes committed changes persistent.

How is it used? Remember: A=All or Nothing, C=Valid State, I=Isolation, D=Data survives commit.

103. What is @Transactional?

What is it? @Transactional tells Spring to execute a method/class within a transaction boundary.

Why is it used? It lets multiple database operations be treated as one unit and manages commit/rollback.

How is it used? Place it commonly on service-layer methods; Spring applies transaction management through proxies/interceptors.

104. Why do we use @Transactional?

What is it? We use it to define transaction boundaries around operations that must succeed or fail together.

Why is it used? It provides atomicity and consistent database updates and automates commit/rollback behavior.

How is it used? A service method containing related database updates is a common place to use it.

Example: Debit and credit should commit together or roll back together.

105. Commit vs Rollback?

What is it? Commit permanently applies transaction changes; rollback reverts changes made by the transaction.

Why is it used? Commit is used after successful work; rollback prevents partial or invalid updates.

How is it used? Success → COMMIT; rollback condition → ROLLBACK.

106. What happens if an exception occurs inside a Transaction?

What is it? Spring decides whether to roll back based on the exception type and transaction configuration.

Why is it used? By default, declarative transactions roll back for unchecked RuntimeException and Error; checked exceptions do not automatically trigger rollback.

How is it used? Use rollbackFor when checked exceptions should also trigger rollback, e.g. @Transactional(rollbackFor = Exception.class).

Example: If credit() throws RuntimeException after debit(), the transaction can roll back both operations.